Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Bilgisayar mimarisi (kısa tekrar)
  - Von Neumann mimarisi: bir işlemci ve bir ana bellek vardır. Programın komutları ve verileri aynı bellekte durur. Ayrıca giriş/çıkış birimleri vardır.
  - İşlemcide aritmetik-mantık birimi (ALU) komutları yürütür (execution). Yazmaçlar (register) işlemci içindeki küçük kayıt alanlarıdır.
    - Program sayacı (program counter, PC): sıradaki komutun bellekteki adresini tutar.
    - Komut yazmacı (instruction register) ve bellek tampon yazmacı (memory buffer register): bellekten gelen komut ve veriler.
    - Genel amaçlı yazmaçlar (AX, BX gibi).
  - Bellek hiyerarşisi: yazmaçlar, önbellek (cache), ana bellek (RAM), disk (HDD/SSD). Yukarı çıktıkça hız artar, aşağı indikçe kapasite artar.
  - Program ana bellekte durur. PC hangi adresi gösterirse o komut getirilip çalıştırılır.
  - Birden çok program aynı anda çalışırken işletim sistemi işlemciyi, belleği ve giriş/çıkış kaynaklarını programlar arasında paylaştırır ve yönetir.
- C programının yapısı
  - Her C programında bir `main` fonksiyonu olmak zorundadır.
  - Kütüphaneler `#include` ile başlık dosyası (header file) üzerinden dahil edilir.
- Veri tipleri
  - Dört temel tip: tam sayı (`int`), tek duyarlıklı ve çift duyarlıklı ondalıklı sayı (`float`, `double`), karakter (`char`). Diziler ve string bunlardan türer.
  - C'de string tipi yoktur, karakter dizisi kullanılır. Boolean tipi de yoktur: yanlış için 0, doğru için 1 kullanılır.
  - Bildirim (declaration) ve tanım (definition) ayrımı derste anlatıldı (düzeltme için "Ders sonrası notlar"a bakın).
- Tam sayılar
  - `int` 32 bittir. `signed` (varsayılan) için 1 bit işarete gider: aralık -2^{31} ile $2^{31}-1$.
  - `unsigned` için işaret biti yoktur: aralık 0 ile $2^{32}-1$.
  - `short` (en az 16 bit), `long` (en az 32 bit), `long long` (en az 64 bit).
  - Neden daha küçük tipler? Gömülü sistemlerde ve eski sistemlerde bellek kısıtlıdır. İşletim sistemi RAM'i programlar arasında paylaştırır. Gereğinden fazla bellek kullanmak diğer programlara sorun çıkarabilir.
  - `sizeof` bir fonksiyon değil, bir operatördür. Derste: `sizeof(int)` 4, `sizeof(short)` 2, `sizeof(long long int)` 8.
- Ondalıklı sayılar: IEEE 754 standardı
  - Sayı bilimsel gösterimle yazılır: işaret biti, üs (exponent) ve kesir (fraction/mantissa).
  - Tek duyarlık (32 bit): 1 işaret + 8 üs + 23 kesir biti. Çift duyarlık (64 bit): 1 + 11 + 52.
  - `float` yaklaşık $\pm 10^{38}$ aralığını kapsar. `sizeof(float)` 4, `sizeof(double)` 8.
  - Örnek: 0.15625 ve 25.68 IEEE 754 hesaplayıcılarında gösterildi.
- Karakterler
  - `char` 1 bayttır, 0-255 arası değer alabilir. ASCII tablosunda her sayı bir sembole karşılık gelir.
  - Windows'ta Alt + sayı ile karakter yazılabilir: 65 `A`, 124 `|`, 48 `0`, 49 `1`.
- Tip dönüşümü (casting)

  - Bir değişkenin değerini başka bir tipe dönüştürmektir. Küçükten büyüğe veya büyükten küçüğe olabilir, ama bilinçli yapılmalıdır. Derleyici her zaman uyarmaz.
  - Örtük (implicit): derleyici kendiliğinden yapar. `int x = 1; double f = 2.3; double y = x + f;` satırında `x` önce 1.0'a dönüşür. Kodun okunuşundan fark edilmez, bu yüzden daha tehlikelidir.
  - Açık (explicit): `(double)x`. Cast operatörünün önceliği toplamadan yüksektir ve sağdan sola birleşir.
  - `char` (1 bayt) değeri `int` (4 bayt) içine konursa kayıp olmaz. Tersi kayıplıdır.
  - `double` değeri `int`'e atanınca ondalık kısım atılır (truncation), yuvarlama yapılmaz. Derste `int t = (int)z;` denendi ve `t` 2 çıktı: "ikiyi aldı, 0.3'lü kısmı attı". `z` 3.3 olsaydı `t` 3 olurdu (gcc ile denendi).
  - Biçim belirteci yanlışsa (ör. `double` için `%d`) anlamsız sayılar görülür.
- Virgül operatörü
  - Soldan sağa çalışır, ifadenin değeri en sağdaki değerdir.
  - `int funct(){ return 2,8,7; }` 7 döndürür.
  - Ondalık sayılarda virgül yerine nokta kullanın: `2,8` C'de 2.8 değildir.
- Arttırma operatörleri
  - `x++` (sonek): ifadenin sonunda arttırılır. `++x` (önek): ifadenin başında arttırılır.
  - Slayttaki örnek: `int x = 3;`, ardından `x++;` ve `++x;` satırları x'i 5 yapar. `y = ++x + 3;` satırında önce x 6 olur, y 9 olur. `z = x++ + 3;` satırında z, x'in eski değeriyle hesaplanır (z = 9), sonra x 7 olur (gcc ile denendi).
  - `printf("%d %d", ++j, --k)` önce değerleri değiştirir: 6 ve 4. `printf("%d %d", j++, k--)` eski değerleri yazar: 5 ve 5, sonraki satırda güncel değerler görülür.
- İlişkisel ve mantıksal operatörler
  - Sonuç doğruysa 1, yanlışsa 0 üretir. `if` bu değere göre akışı yönlendirir.
  - VE (&&): kümelerde kesişim gibi, ikisi de doğruysa 1. VEYA (||): birleşim gibi, en az biri doğruysa 1. DEĞİL (!).
  - VEYA'da sol taraf doğruysa sağ tarafa bakılmaz.
  - Örnek: "devamsızlığı 10 saatten az VE ortalaması 65'ten büyük olan geçer" ile "VEYA" arasındaki anlam farkı.
- `printf` biçimlendirme
  - `%d`/`%i` tam sayı, `%f` float, `%lf` double, `%c` karakter, `%u` işaretsiz, `%s` string.
  - `%N1.N2f`: N2 virgülden sonraki basamak sayısıdır. N1 ile ilgili açıklama için "Ders sonrası notlar"a bakın.
  - `%03d`: önünü sıfırla doldurur (zero padding). Doldurma karakteri olarak yalnızca 0 kullanılabilir.
  - Kaçış karakterleri: `\n` yeni satır, `\t` sekme, `\v` dikey sekme, `\r` satır başı.
- Tam sayı bölmesi
  - İki tam sayı bölünürse sonuç tam sayıdır, ondalık kısım atılır: `16/2` 8, `16/3` 5, `15/2` 7 (7.5 yuvarlanıp 8 olmaz).
  - Operandlardan biri ondalıklıysa sonuç ondalıklı olur: `15/2.0` ve `15.0/2` 7.5. `(double)15/2` de 7.5 verir, çünkü cast bölmeden önce yapılır.
  - `(double)(15/2)` ise önce tam sayı bölmesi yapar: 7.0 (gcc ile denendi). Derste ekranda 0.0 görüldü. Bu, büyük olasılıkla `printf("%lf", 15/2)` gibi `int` bir değerin `%lf` ile yazdırılmasından gelir: gcc bu satır için uyarı verdi ve 0.000000 yazdı.
  - Slayt örneği: `b = 15/2;` ile `%d` 7, `%3d` " 7", `%03d` "007". `c = 15.3; d = c/3;` ile `%3.2f` 5.10.
- Atama: sol değer ve sağ değer
  - Atamada her zaman sağdaki değer sola kopyalanır.
  - Sol taraf (L-value) yalnızca değişken olabilir. Sağ taraf (R-value) değişken, sabit literal (3, 5) veya sabit (const) olabilir. `5 = answer` veya sol tarafta operatör kullanmak hatalıdır.
- Kapsamlı örnek (`hafta2_impl_expl_cast_operators.c`)
  - `printf("%d %f", -3/4, -3/4.0)` 0 ve -0.75 yazar.
  - `-3/4 + 2.5` 2.5'tir (tam sayı bölmesi 0 verir). `-3/4.0 + 2.5` 1.75'tir.
  - `1 + 2.5` 3.5'tir: küçük tip büyük tipe dönüşür.
  - `10u - 15` `%u` ile yazılınca 4294967291 gibi çok büyük bir sayı görülür: -5'in ikiye tümleyen (two's complement) bit dizisi işaretsiz okunur.
  - `float f = 15; int j = 5;` ile `j + f + 2.5` ifadesi `%2.2f` ile 22.50 yazar.
  - `j = j + f + 2.5;` sonucu 22 olur (23 değil, kesilir). `j = 2.5;` 2, `j = 999.888888;`

999.
  - `INT_MAX` $2^{31} - 1$'dir. İkiye tümleyende sıfır tek ve pozitif taraftadır, bu yüzden pozitif taraf bir eksik biter.
  - `f = k/j` (k=3, j=2) 1.0, `f = (float)k/j` 1.5.
- İkiye tümleyen (two's complement)
  - Negatif sayı bulmak için bitler terslenir (birler tümleyeni, one's complement), sonra 1 eklenir.
  - Örnek: 86 = `1010110`. -86 için terslenip 1 eklenir: son bayt `10101010`, kalan bitler hep 1.
  - Birler tümleyeninde +0 ve -0 vardır, ikiye tümleyende tek sıfır vardır.
- Önek/sonek tartışması: `n = m++ * m++;` (m=1)
  - Derste Dev-C++'ta n = 2, m = 3 çıktı. Hoca operatör önceliğine göre 6 bekliyordu. Assembly çıktısına bakarak derleyicinin önce eski değeri bir yazmaca alıp sonra belleği arttırdığını gösterdi. Bunu araştırıp döneceğini söyledi (cevap için "Ders sonrası notlar"a bakın).
- Numaralandırma (`enum`)
  - Haftanın günleri gibi sabit seçenekleri isimlerle, arka planda tam sayılarla yönetir. String karşılaştırmaya göre daha güvenlidir.
  - İlk değer verilmezse 0'dan başlar. Bir değere ilk değer verilirse sonrakiler birer artar: 3, 4, 5, sonra -6 verilirse -5, sonra 10 verilirse 11.
  - `switch` ile birlikte kullanışlıdır.
- Ömür (duration)
  - Yerel değişkenler ve sabit uzunluklu yerel diziler yığında (stack) oluşur. Fonksiyon bitince bu alan serbest kalır.
  - `malloc` ile alınan bellek öbekte (heap) oluşur. Fonksiyon bitse de `free` edilene kadar yaşar. `free` edilmezse bellek sızıntısı (memory leak) olur.
  - Ayrıntısı `malloc` ve fonksiyonlar konusunda işlenecek.

Hocanın Özellikle Vurguladığı Kısımlar

- Örtük dönüşümler daha tehlikelidir, çünkü koddan görünmez.
- Tam sayı bölmesinde ondalık kısım kesilir, yuvarlama yoktur. Ondalıklı sayıyı tam sayıya atarken de kesilir.
- Bu tür operatör önceliği ve tip dönüşümü soruları sınavlarda sorulabilir. Ama `m++ * m++` gibi kafa karıştırıcı ifadeler sorulmaz.
- Biçim belirtecini değişkenin tipine uygun seçin.
- Ondalık sayılarda virgül değil nokta kullanın.

Kısa Tekrar Notları

- `sizeof`: int 4, short 2, long long 8, float 4, double 8.
- `int`: $-2^{31} \dots 2^{31} - 1$, `unsigned int`: $0 \dots 2^{32} - 1$.
- IEEE 754: float 1+8+23, double 1+11+52.
- `15/2` = 7, `15/2.0` = 7.5, `(double)15/2` = 7.5, `(int)3.9` = 3.
- `return 2,8,7;` 7 döndürür.
- `%03d` sıfırla doldurur, `%.2f` iki ondalık basamak.
- `enum {A=3, B, C, D=-6, E, F=10, G}` → 3, 4, 5, -6, -5, 10, 11.

Detaylı Açıklamalar

`n = m++ * m++` neden tutarsız?

- C standardına göre aynı değişkeni iki sıralama noktası (sequence point) arasında iki kez değiştirmek **tanımsız davranıştır** (undefined behavior). Bu bir öncelik sorusu değildir: öncelik yalnızca hangi operandın hangi operatöre ait olduğunu belirler, hesaplama sırasını belirlemez.
- Bu yüzden derleyiciye, ayara ve mimariye göre 2, 6 veya başka bir sonuç çıkabilir. Derste görülen 2 (n) ve 3 (m) GCC'nin bir tercihidir.
- `gcc -Wall` bu satır için uyarı verir: `operation on 'm' may be undefined [-Wsequence-point]`. Derleyici uyarılarını açık tutun.

- Aynı kural `printf("%d %d", j++, j)` gibi bir ifadede de geçerlidir. Fonksiyon argümanlarının hesaplanma sırası da belirsizdir.

Ders sonrası notlar

- **Kodların kontrolü:** Dersteki örnekler ders sonrasında `gcc` ile derlenip çalıştırıldı. `sizeof` değerleri, tam sayı bölmeleri, `-3/4 + 2.5` = 2.5, `-3/4.0 + 2.5` = 1.75, `10u - 15` = 4294967291, `j + f + 2.5` = 22.50, kesme örnekleri, `funct()` = 7 ve `enum` değerleri derstekiyle aynıdır.
- **`int` aralığı slaytta:** Slaytta signed aralığı "-2^{31} ile 2^{31}", unsigned aralığı "0 ile 2^{32}" yazıyor. Doğrusu $-2^{31} \dots 2^{31}-1$ ve $0 \dots 2^{32}-1$'dir. Hoca derste "-1" eksikliğini düzeltti.
- **Bildirim ve tanım:** Derste "değer atanmayan `int x;` yalnızca bildirimdir, değer atanınca tanım olur" dendi. C'de fonksiyon içindeki `int x;` hem bildirim hem tanımdır: bellek ayrılır, sadece başlangıç değeri yoktur (içinde rastgele değer olabilir). Yalnızca bildirim örneği `extern int x;` veya bir fonksiyon prototipidir. `int x = 5;` ilk değer verilmiş (initialized) tanımdır.
- **Başlık dosyaları:** Başlık dosyaları fonksiyonların gövdesini değil, bildirimlerini (prototiplerini) içerir. Gövdeler derlenmiş kütüphanelerdedir ve bağlama (linking) aşamasında programa eklenir.
- **`printf` çıkışı:** `printf` standart girişe değil standart çıkışa (stdout) yazar.
- **`%N1.N2f` biçimi:** N1, virgülden önceki basamak sayısı değil, alanın **toplam en küçük genişliğidir** (nokta ve ondalık basamaklar dahil). Sayı daha uzunsa kesilmez. Örnek: `printf("%2.3f", 4524.3659974)` ve `%8.3f` ikisi de "4524.366" yazar. `%10.3f` " 4524.366", `%010.3f` "004524.366" yazar. Önüne sıfır koymak için `0` bayrağı gerekir.
- **Sonek arttırma ve performans:** Büyük olasılıkla derste `x++`'ın assembly seviyesinde daha hızlı olduğu söylendi. Tam sayı tipleri için derleyici `x++` ve `++x`'i (sonucu kullanılmadığında) aynı koda çevirir (`gcc -O0 -S` ile denendi). Bir fark varsa kavramsal olarak `++x` geçici kopya gerektirmediği için daha basittir. `for` döngüsünde hangisini kullandığınız sonucu değiştirmez.
- **IEEE 754 üssü:** Sayı ikilik tabanda normalize edilir: üs 10'un değil 2'nin kuvvetidir. Örnek: $0.15625 = 1.01_2 \times 2^{-3}$. Tek duyarlıkta üs alanına $-3 + 127 = 124$ yazılır.
- **ASCII aralığı:** Standart ASCII 7 bittir (0-127). 128-255 arası genişletilmiş (extended) kod sayfalarıdır ve sisteme göre değişir.
- **Boolean:** C99'dan beri `_Bool` tipi ve `<stdbool.h>` ile `bool`, `true`, `false` vardır. Ders kapsamında 0/1 kullanılır.
- **Java karşılaştırması:** Java'da `double`'dan `int`'e örtük atama derleme hatası verir, ama açık cast `((int)z)` hata vermez. Derste açık cast denendiği için hata görülmedi.
- **Virgül:** `double yy = 2.3, yyy = yy + 1.0;` satırındaki virgül, virgül operatörü değil bildirim ayırıcısıdır. Yine de bildirimler soldan sağa yapılır, bu yüzden `yyy` doğru hesaplanır. `return 2,8,7;` ise gerçek virgül operatörüdür.
- **Ömür (duration) terimleri:** Derste öbekteki bellek için "static duration" denildi. C standardında üç tür vardır: yerel değişkenler için otomatik (automatic), global ve `static` değişkenler için statik (static), `malloc` ile alınan bellek için ayrılmış/dinamik (allocated) ömür. `malloc` belleği "dinamik"tir.
- **Biçim belirteci uyuşmazlığı:** `double` bir değeri `%d` ile yazdırmak da tanımsız davranıştır. Görülen sayı bit dizisinin yanlış yorumlanmasından gelir ve sistemden sisteme değişir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.